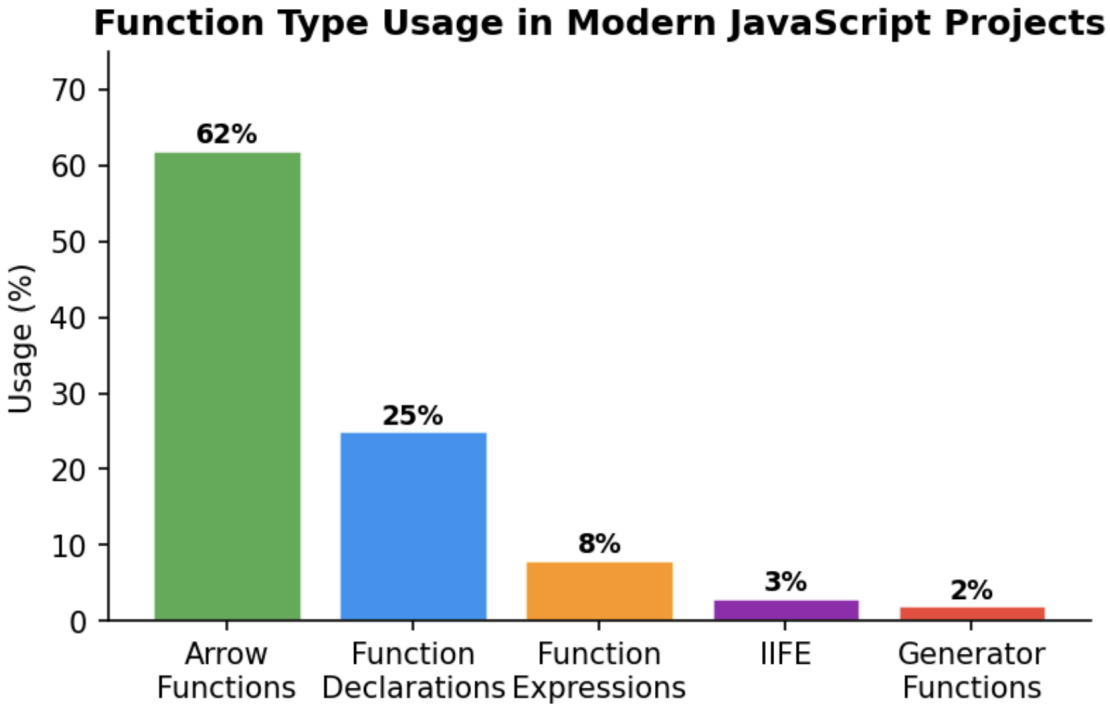
Functions, Scope, and Closures

Functions are one of the fundamental building blocks in JavaScript. A function is a reusable block of code designed to perform a particular task. In this lesson, we explore function declarations, expressions, arrow functions, scope rules, and closures.



1. Function Declarations vs Expressions

A function declaration defines a named function using the 'function' keyword. It is hoisted to the top of its scope, meaning it can be called before it appears in the code. A function expression assigns an anonymous (or named) function to a variable. Unlike declarations, expressions are not hoisted.

Example:

```
// Declaration
function greet(name) { return `Hello, ${name}!`; }
```

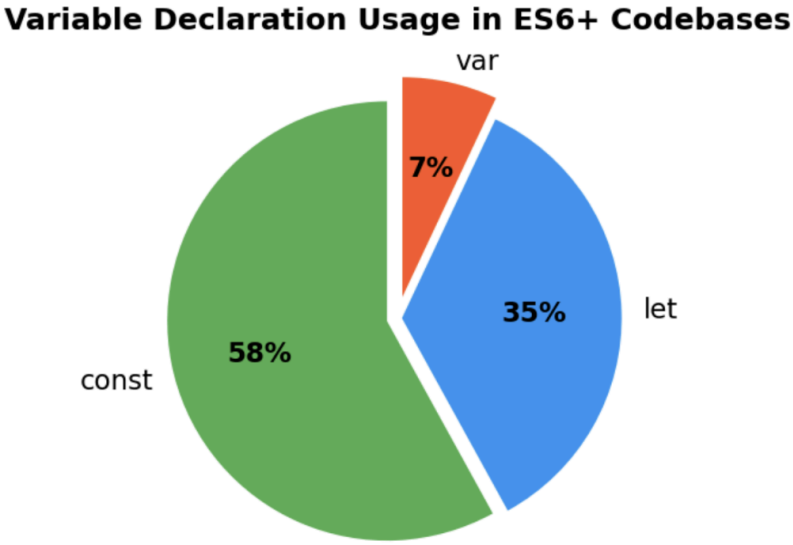
```
// Expression
const greet = function(name) { return `Hello, ${name}!`; };
// Arrow Function (ES6+)
const greet = (name) => `Hello, ${name}!`;
```

2. Understanding Scope

Scope determines the accessibility of variables. JavaScript has three types of scope:

- Global Scope: Variables declared outside any function or block
- Function Scope: Variables declared inside a function (var)
- Block Scope: Variables declared inside a block with let/const

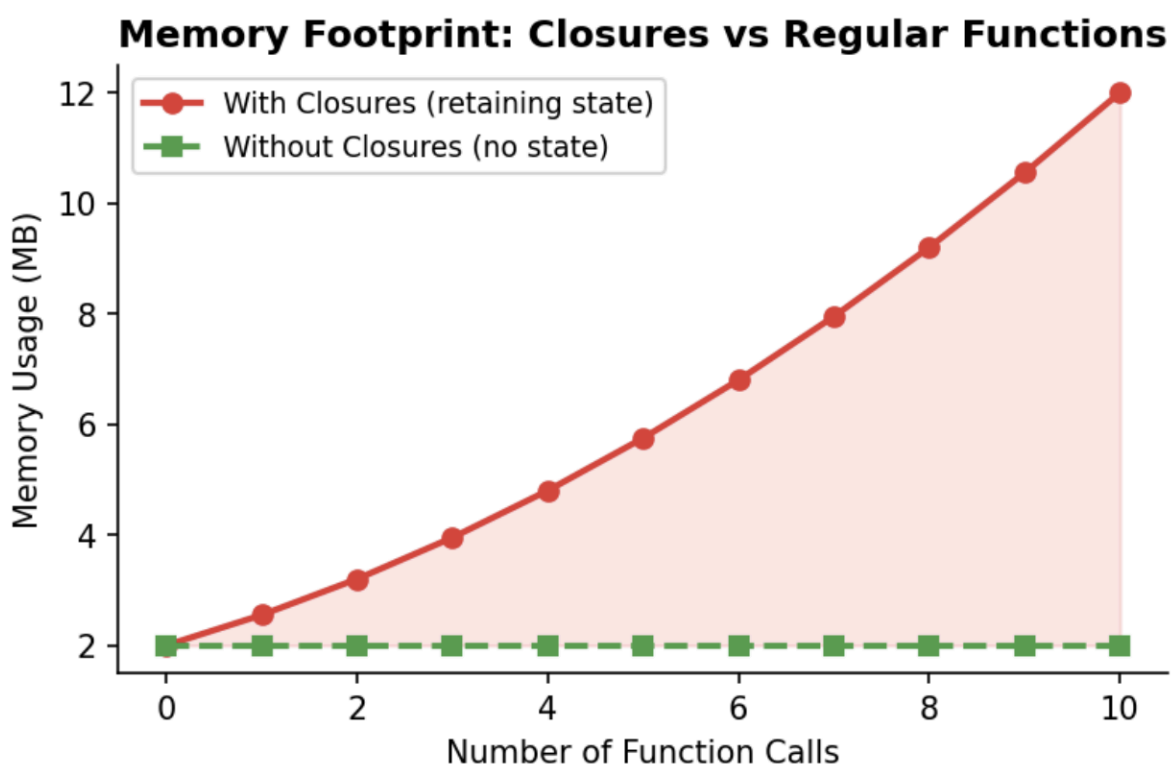
The scope chain allows inner functions to access variables from outer scopes, but not the other way around.



3. Closures

A closure is a function that retains access to its outer (enclosing) function's variables even after the outer function has returned. Closures are created every time a function is created.

Example:

```
function createCounter() {
  let count = 0;
  return {
    increment: () => ++count,
    getCount: () => count
  };
}
const counter = createCounter();
counter.increment(); // 1
counter.increment(); // 2
counter.getCount();  // 2
```

Closures are widely used for data privacy, factory functions, memoization, and maintaining state in functional programming patterns